GIT · GITHUB · GITHUB PAGES

Ship your project to GitHub

Create the repository, deploy the site, and run the live demo. Every step comes with the command and a plain-language explanation of what it does.

IN THIS GUIDE

01	Setup	Git, identity, terminal
02	Create the repository	From local folder to GitHub
03	Deploy with GitHub Pages	Get a public link
04	Live demo	Edit, push, refresh

SECTION 01

Setup

Three quick checks before you start. You only need to do this once per computer.

01 Check that Git is installed

```
$ git --version
```

WHAT IT DOES

Prints the installed Git version. If you see a version number, you are ready. If not, install Git from git-scm.com.

02 Tell Git who you are

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
```

WHAT IT DOES

Saves your name and email so every commit you make is signed with them. The `--global` flag applies the setting to all projects on this computer.

03 Open the terminal in VS Code

Open your project folder in VS Code, then press `Ctrl` + `` ` `` to open the integrated terminal. Every command in this guide runs there.

SECTION 02

Create the repository

Turn your project folder into a Git repository, save a first snapshot, and upload it to GitHub.



01 Create a .gitignore file

.GITIGNORE

```
node_modules
.env
```

WHAT IT DOES

Tells Git which files to skip. `node_modules` is huge and can be rebuilt with one command. `.env` holds secrets like passwords and API keys that must never be published.

02 Initialize the repository

```
$ git init
```

WHAT IT DOES

Makes Git start tracking this folder. It creates a hidden `.git` folder where the project history is stored.

03 Check the status of your files

```
$ git status
```

WHAT IT DOES

Lists the files Git can see. Files shown in red under *Untracked files* are visible to Git but not saved yet.

04 Stage your files

```
$ git add .
```

WHAT IT DOES

Moves every change into the staging area, ready to be saved. The dot means "everything". Warnings about LF and CRLF on Windows are normal and can be ignored.

05 Save your first commit

```
$ git commit -m "first commit"
```

WHAT IT DOES

Saves a snapshot of the staged files together with a message that describes the change. Git then prints how many files were saved.

06 Name the branch main

```
$ git branch -M main
```

WHAT IT DOES

Renames the current branch from `master` to `main`, which is the default name on GitHub.

07 Create a repository on GitHub

- Sign in at **github.com**.
- Click the **+** icon at the top and choose **New repository**.
- Enter a repository name.
- Do **not** tick "Add a README" or ".gitignore". Your files already exist locally.
- Click **Create repository** and copy the URL.

08 Connect your folder to GitHub

```
$ git remote add origin https://github.com/username/my-project.git
```

WHAT IT DOES

Tells Git that a copy of this project lives online and gives it the name `origin`. Replace the URL with the one you copied.

09 Push your code

```
$ git push -u origin main
```

WHAT IT DOES

Uploads your commits to GitHub. The first time, a browser window opens so you can sign in. The `-u` flag remembers the connection, so next time you only need `git push`.

SECTION 03

Deploy with GitHub Pages

GitHub Pages turns your repository into a real website with a public link. Your friends see the site, not the code. It is free.

01 Open the Pages settings

On your repository page click **Settings**, then choose **Pages** in the left sidebar.

02 Choose what to publish

Under **Source** select **Deploy from a branch**. Under **Branch** select `main` and the folder `/ (root)`, then click **Save**.

03 Wait a minute and copy the link

After one or two minutes, refresh the page. A message reading "Your site is live at" appears with your link. It looks like this:

```
https://username.github.io/repo-name/
```

> IMPORTANT
>
> On the free plan the repository must be **Public**, which means anyone can read the code. Make sure no passwords or private files are included.

The index.html rule

GitHub Pages looks for a file named `index.html` as the home page. If your main file has another name, the plain link returns a **404**. Rename the file and push the change:

```
$ git mv page.html index.html
$ git commit -m "rename to index.html"
$ git push
```

> WHAT IT DOES
>
> `git mv` renames the file and stages the change in one step. After the push, wait a minute and the plain link opens your site.

SECTION 04

Live demo

Show that any change reaches the live site with three commands. This is the loop you will use every day.



01 Change something in the page

Edit a heading or a line of text in `index.html` and save the file.

02 Push the change

```
$ git add .
$ git commit -m "update title"
$ git push
```

> WHAT IT DOES
>
> The first command stages the edit, the second saves it with a message, and the third uploads it. GitHub Pages then republishes the site automatically.

03 Open the site and refresh

Wait about a minute, then open the link and press `Ctrl` + `F5` (or use a private window). The change is now live.

If something goes wrong

PROBLEM	FIX
The page shows 404	There is no `index.html`. Rename your main file, or open the link with the file name at the end.
An image does not appear	The HTML points to a path on your computer such as `C:\...`. Use a relative path, which is just the file name.
The change is not visible	Wait one or two minutes, then hard refresh with `Ctrl` + `F5`.